

Savitribai Phule Pune University

Third Year Information Technology (2019 Course)

314458: Laboratory Practice-II (Cloud Computing)

ORAL EXAMINATION QUESTION BANK

All 8 Assignments / External Viva Questions & Answers

Assignment 1: Google App Engine – Hello World & Simple Web Apps

Key Oral Questions & Answers

Q1. What is Google App Engine (GAE)?

Ans: Google App Engine is a Platform-as-a-Service (PaaS) offering from Google Cloud that allows developers to build and deploy applications on Google's infrastructure. It automatically manages hardware and networking, scaling resources as needed. It supports languages like Python, Java, Node.js, Go, etc.

Q2. What are the two environments in GAE?

Ans: GAE offers two environments: (1) Standard Environment – runs in a sandboxed environment with support for specific language runtimes; fast scaling including to zero. (2) Flexible Environment – runs inside Docker containers on Google Compute Engine VMs; supports more runtimes and libraries.

Q3. What is app.yaml and why is it important?

Ans: app.yaml is the configuration file for a GAE application. It specifies the runtime (e.g., python39), URL handlers (url, script), environment variables, instance class, scaling type, and other settings. Without it, GAE cannot deploy the application.

Q4. How do you deploy an application to Google App Engine?

Ans: Using the Google Cloud SDK, you run the command 'gcloud app deploy' in the project directory containing app.yaml. Before that, you must authenticate using 'gcloud auth login' and set the project using 'gcloud config set project [PROJECT_ID]'.

Q5. What is the difference between PaaS, IaaS, and SaaS?

Ans: IaaS (Infrastructure as a Service) provides virtualized computing resources like VMs, storage, networking (e.g., AWS EC2). PaaS (Platform as a Service) provides a platform with runtime environment, databases, and middleware to develop and deploy apps without managing infrastructure (e.g., GAE). SaaS (Software as a Service) delivers ready-to-use software over the internet (e.g., Gmail, Office 365).

Q6. What scaling options does GAE provide?

Ans: GAE provides three scaling options: (1) Automatic Scaling – scales based on request rate, response latency, and other metrics. (2) Basic Scaling – creates instances when the app receives requests and shuts them down when idle. (3) Manual Scaling – runs a fixed number of instances.

Q7. What is the Google Cloud SDK?

Ans: Google Cloud SDK is a set of command-line tools (gcloud, gsutil, bq) used to interact with Google Cloud services. It is used to manage GAE deployments, create and manage GCP resources, and authenticate with Google accounts.

Q8. What is a 'Hello World' application in GAE using Python?

Ans: A simple Hello World app in GAE Python consists of: (1) main.py – uses a framework like Flask or webapp2 to return 'Hello, World!' for the root URL. (2) app.yaml – specifies runtime: python39, with a URL handler pointing to main.app. (3) requirements.txt – lists Flask as a dependency.

Q9. What are handlers in app.yaml?

Ans: Handlers in app.yaml define URL routing rules. Each handler maps a URL pattern to either a script (Python WSGI app), a static file, or a static directory. For example, url: /* script: auto routes all URLs to the app's main handler.

Q10. What is the purpose of requirements.txt in a GAE Python app?

Ans: requirements.txt lists all Python package dependencies for the application (e.g., Flask, requests). During deployment, GAE reads this file and installs the required packages into the runtime environment automatically.

Assignment 2: GAE Launcher – Launch Web Applications

Key Oral Questions & Answers

Q1. What is the Google App Engine Launcher?

Ans: GAE Launcher is a desktop GUI tool (available for Windows and macOS) used to manage local development of GAE applications. It allows developers to create, run, deploy, and browse GAE applications from a graphical interface instead of command-line tools.

Q2. What is the purpose of the local development server in GAE?

Ans: The local development server (`dev_appserver.py`) simulates the GAE environment on your local machine. It allows testing and debugging applications before deploying to production. It mimics GAE APIs, the datastore, and URL routing behavior.

Q3. What port does the GAE development server run on by default?

Ans: By default, the GAE development server runs on port 8080 for the application. The admin console is accessible on port 8000. These ports can be configured using the `--port` and `--admin_port` flags.

Q4. What are the steps to launch a web application using GAE Launcher?

Ans: Steps: (1) Open GAE Launcher. (2) Click 'Add Existing Application' or 'Create New Application'. (3) Browse to your application folder. (4) Click 'Run' to start the local server. (5) Click 'Browse' to open the app in a browser at `localhost:8080`.

Q5. What is the difference between 'Run' and 'Deploy' in GAE Launcher?

Ans: 'Run' starts the application on the local development server (`localhost`) for testing purposes. 'Deploy' uploads the application to Google's production servers, making it publicly accessible on `appspot.com`.

Q6. What is webapp2 framework in GAE?

Ans: `webapp2` is a lightweight Python web framework designed for GAE Standard Environment. It maps URL patterns to Python handler classes. Each handler inherits from `webapp2.RequestHandler` and implements `get()` or `post()` methods to handle HTTP requests.

Q7. How do you pass parameters in a GAE web application URL?

Ans: URL parameters can be passed as query strings (e.g., `/greet?name=John`) and accessed via `self.request.get('name')` in the handler. Alternatively, URL path parameters can be captured using `regex` in the URL mapping (e.g., `/user/(.+)`).

Q8. What is the GAE Admin Console?

Ans: The GAE Admin Console (accessible at `localhost:8000` during development or `console.cloud.google.com` in production) provides tools to view application logs, monitor datastore entities, manage task queues, configure settings, and view version history.

Q9. What file structure is required for a basic GAE Python application?

Ans: A basic GAE Python app requires: `app.yaml` (configuration), `main.py` (application code / WSGI handler), `requirements.txt` (dependencies), and optionally `static/` folder (for static files like HTML, CSS, JS) and `templates/` folder (for Jinja2 templates).

Q10. What is WSGI and how does GAE use it?

Ans: WSGI (Web Server Gateway Interface) is a standard interface between web servers and Python web applications. GAE uses WSGI to pass incoming HTTP requests to the Python application. In `app.yaml`, `'script: auto'` lets GAE find the WSGI app automatically, or you can specify `'script: main.app'`.

Assignment 3: CloudSim – Simulate Cloud & Custom Scheduling Algorithm

Key Oral Questions & Answers

Q1. What is CloudSim?

Ans: CloudSim is an open-source simulation toolkit developed by the CLOUDS Lab at the University of Melbourne. It allows researchers and practitioners to model and simulate cloud computing environments, data centers, virtual machines, cloudlets (tasks), and resource scheduling policies in a controlled environment without needing actual cloud infrastructure.

Q2. What are the main components/entities in CloudSim?

Ans: Main entities: (1) CloudSim core – simulation engine. (2) Datacenter – models physical cloud infrastructure. (3) DatacenterBroker – acts as a user/cloud mediator that submits VMs and cloudlets. (4) VirtualMachine (Vm) – represents a virtual machine with CPU, RAM, BW. (5) Cloudlet – represents a cloud task with length and file size. (6) CloudletScheduler – defines how tasks are scheduled on a VM.

Q3. What is a Cloudlet in CloudSim?

Ans: A Cloudlet is a unit of work or task submitted to the cloud. It is characterized by: cloudlet length (in MI – Million Instructions), file input/output size, number of CPUs required, and utilization models for CPU, RAM, and bandwidth. After execution, it returns metrics like execution time and status.

Q4. What scheduling algorithms are available in CloudSim by default?

Ans: CloudSim provides: (1) CloudletSchedulerTimeShared – multiple cloudlets share VM CPU time simultaneously. (2) CloudletSchedulerSpaceShared – cloudlets are queued and executed one at a time (FCFS). (3) VmSchedulerTimeShared – VMs share host CPU resources. (4) VmSchedulerSpaceShared – each VM gets dedicated CPU cores.

Q5. How do you implement a custom scheduling algorithm in CloudSim?

Ans: To implement a custom algorithm: (1) Create a new class extending CloudletScheduler or DatacenterBroker. (2) Override the relevant method (e.g., cloudletSubmit(), processCloudletReturn(), or submitCloudlets()). (3) Implement your logic (e.g., priority-based, round robin, SJF). (4) Use this custom class when creating VM or broker objects.

Q6. What is the difference between Time Shared and Space Shared scheduling?

Ans: Time Shared scheduling allows multiple cloudlets/VMs to share the CPU simultaneously using time-slicing, similar to multithreading. It increases utilization but introduces context-switching overhead. Space Shared scheduling assigns dedicated CPU cores to each task; tasks execute one at a time in FIFO order. It is simpler but may leave resources idle while other tasks wait.

Q7. What is a DatacenterBroker in CloudSim?

Ans: DatacenterBroker is an intermediary between users and cloud data centers. It: submits VM creation requests to data centers, submits cloudlets for execution, maps cloudlets to VMs, and collects results. Custom broker classes can implement intelligent VM selection and cloudlet scheduling policies.

Q8. What Java version and libraries are needed to run CloudSim?

Ans: CloudSim requires Java Development Kit (JDK) 7 or higher. The CloudSim JAR files (cloudsim-x.x.jar and cloudsim-examples-x.x.jar) must be added to the project's classpath. The org.cloudbus.cloudsim package contains all core classes.

Q9. How do you measure the performance of a scheduling algorithm in CloudSim?

Ans: Performance is measured using: (1) Makespan – total time from start to completion of all cloudlets. (2) Average turnaround time – average time for a cloudlet from submission to completion.

(3) Resource utilization – percentage of VM CPU time used. (4) Waiting time – time cloudlets spend in queue. (5) Throughput – number of cloudlets completed per unit time.

Q10. What is a VirtualMachine object in CloudSim and what parameters define it?

Ans: A Vm object in CloudSim is defined by: vmId (unique ID), userId (broker ID), mips (processing speed), numberOfPes (number of CPUs), ram (RAM in MB), bw (bandwidth), size (storage in MB), vmm (virtual machine monitor, e.g., 'Xen'), and a CloudletScheduler instance.

Assignment 4: File Transfer Between Virtual Machines

Key Oral Questions & Answers

Q1. What methods can be used to transfer files between virtual machines?

Ans: Common methods include: (1) SCP (Secure Copy Protocol) – uses SSH for secure file transfer. (2) SFTP (Secure FTP) – interactive SSH-based file transfer. (3) rsync – efficient synchronization tool supporting incremental transfer. (4) Shared storage/network file systems (NFS, SMB/CIFS). (5) Cloud storage (S3, GCS buckets) as intermediary. (6) HTTP/FTP servers. (7) scp/sftp with key-based authentication.

Q2. What is SCP and how is it used?

Ans: SCP (Secure Copy Protocol) is a command-line utility that uses SSH to securely transfer files between hosts. Command: 'scp [options] source_file user@destination_ip:/destination_path'. Example: 'scp file.txt ubuntu@192.168.1.2:/home/ubuntu/'. The -r flag copies directories recursively, -i specifies the identity/private key file.

Q3. What is the difference between SCP and SFTP?

Ans: SCP is non-interactive and used for scripted or one-time file transfers. SFTP (SSH File Transfer Protocol) provides an interactive session with commands like ls, cd, put, get, allowing browsing and multiple operations. SFTP also supports resuming interrupted transfers. Both use SSH for encryption and authentication.

Q4. What is rsync and what are its advantages over SCP?

Ans: rsync is a fast file synchronization tool that transfers only the changed parts of files (delta transfer), reducing bandwidth usage. It supports compression (-z), preserves permissions (-a), and can continue interrupted transfers. It is ideal for regular backups and syncing large directories between VMs.

Q5. What is SSH key-based authentication and why is it preferred?

Ans: SSH key-based authentication uses a cryptographic key pair (public and private keys) instead of passwords. The public key is placed on the remote server (~/.ssh/authorized_keys) and the private key stays on the local machine. It is more secure than passwords (resistant to brute-force attacks) and enables automated/scripted transfers without manual password entry.

Q6. How do you set up SSH between two VMs for file transfer?

Ans: Steps: (1) Generate SSH key pair on source VM using 'ssh-keygen'. (2) Copy public key to destination VM using 'ssh-copy-id user@dest_ip'. (3) Verify connection: 'ssh user@dest_ip'. (4) Transfer files using 'scp sourcefile user@dest_ip:/path'. (5) Ensure port 22 is open in the security group/firewall rules.

Q7. What is NFS and when would you use it for VM file sharing?

Ans: NFS (Network File System) is a distributed file system protocol that allows a server to share directories over a network so that clients can mount them as local file systems. It is suitable when multiple VMs need continuous shared access to the same files, like a common data directory. Setup involves installing nfs-server on the host, configuring /etc/exports, and mounting on clients.

Q8. What are security considerations when transferring files between VMs?

Ans: Security considerations: (1) Always use encrypted protocols (SCP, SFTP, HTTPS) – avoid plain FTP/Telnet. (2) Use SSH key-based authentication, not passwords. (3) Restrict SSH access by IP in firewall/security groups. (4) Change default SSH port from 22. (5) Validate file integrity using checksums (MD5/SHA256) after transfer. (6) Use VPN for transfers between VMs in different networks/clouds.

Q9. How do you verify that a file was transferred correctly?

Ans: File integrity can be verified using checksums: (1) On source: 'md5sum filename' or 'sha256sum filename'. (2) Transfer the checksum value along with the file. (3) On destination: run the same command and compare. If checksums match, the file was transferred without corruption.

Q10. What is the role of security groups/firewall rules in VM file transfer?

Ans: Security groups (in cloud environments like AWS/OpenStack) act as virtual firewalls controlling inbound and outbound traffic for VMs. For file transfer via SSH/SCP/SFTP, port 22 must be open for inbound traffic. For HTTP/FTP-based transfer, ports 80/443/21 must be opened. Restricting access to specific IP ranges adds an additional security layer.

Assignment 5: Launch Virtual Machine Using TryStack (OpenStack)

Key Oral Questions & Answers

Q1. What is OpenStack?

Ans: OpenStack is a free, open-source cloud computing platform used to build and manage both public and private clouds. It provides IaaS by managing compute (Nova), networking (Neutron), storage (Swift/Cinder), identity (Keystone), image (Glance), and dashboard (Horizon) services. TryStack is a free public OpenStack demo environment for learning.

Q2. What is TryStack?

Ans: TryStack (trystack.org) was a free OpenStack sandbox environment provided for users to experiment with OpenStack without setting up their own infrastructure. Users could log in via Facebook, create instances, and practice OpenStack operations. It offered limited resources for educational purposes.

Q3. What are the key components/services of OpenStack?

Ans: Key services: (1) Nova – Compute service for managing VMs. (2) Neutron – Networking service for managing virtual networks. (3) Cinder – Block storage service. (4) Swift – Object storage service. (5) Glance – Image service for VM images. (6) Keystone – Identity and authentication service. (7) Horizon – Web-based dashboard. (8) Heat – Orchestration service.

Q4. What is Nova in OpenStack?

Ans: Nova is the compute component of OpenStack responsible for provisioning and managing virtual machine instances. It interacts with hypervisors (KVM, Xen, VMware) through libvirt to create, resize, suspend, and terminate VMs. Nova-api receives requests, nova-scheduler decides which host to run the VM on, and nova-compute creates the VM.

Q5. What is a flavor in OpenStack?

Ans: A flavor defines the virtual hardware specifications for a VM instance, including number of vCPUs, RAM size, root disk size, and sometimes ephemeral disk and swap space. Examples: m1.tiny (1 vCPU, 512MB RAM, 1GB disk), m1.small (1 vCPU, 2GB RAM, 20GB disk). Users select a flavor when launching an instance.

Q6. What is a Security Group in OpenStack?

Ans: A Security Group is a set of firewall rules (IP filter rules) that control inbound and outbound network traffic to VM instances. Rules specify protocol (TCP/UDP/ICMP), port range, and source/destination IP. By default, all inbound traffic is blocked; you must explicitly add rules (e.g., allow SSH on port 22, allow ICMP for ping).

Q7. What is a Key Pair in OpenStack and why is it needed?

Ans: A Key Pair consists of a public-private SSH key pair used for secure, passwordless access to VM instances. The public key is injected into the VM during launch (stored in ~/.ssh/authorized_keys). The user retains the private key to authenticate via SSH. It is required for Linux instances that have no default password login.

Q8. What is the difference between floating IP and fixed IP in OpenStack?

Ans: Fixed IP is the private IP address automatically assigned to a VM from the internal network. It is used for communication within the cloud. Floating IP is a public IP address that can be associated with a VM to allow access from external networks (internet). Floating IPs can be dynamically assigned, disassociated, and reassigned between instances.

Q9. How do you launch a VM instance in OpenStack Horizon dashboard?

Ans: Steps: (1) Login to Horizon. (2) Go to Compute > Instances > Launch Instance. (3) Enter instance name. (4) Select source (image or snapshot). (5) Choose a flavor. (6) Configure networking. (7) Add

security groups. (8) Select or create a key pair. (9) Click Launch. The instance will go through BUILD -> ACTIVE states.

Q10. What is the difference between public cloud, private cloud, and hybrid cloud?

Ans: Public cloud: services provided over the internet by third-party providers (AWS, GCP, Azure); shared infrastructure; pay-per-use model. Private cloud: cloud infrastructure operated solely for one organization; greater control, security, and customization; can be on-premises (e.g., OpenStack on private servers). Hybrid cloud: combination of public and private clouds, allowing workloads to move between them for flexibility.

Assignment 6: Design and Deploy Web Application in PaaS Environment

Key Oral Questions & Answers

Q1. What is PaaS and give examples?

Ans: Platform as a Service (PaaS) is a cloud service model that provides a complete development and deployment environment in the cloud. It includes servers, storage, networking, middleware, development tools, and databases. Developers can build, test, and deploy applications without managing underlying infrastructure. Examples: Google App Engine, Heroku, Microsoft Azure App Service, AWS Elastic Beanstalk, Red Hat OpenShift.

Q2. What are the advantages of deploying a web application in PaaS?

Ans: Advantages: (1) No infrastructure management – platform handles OS, runtime, and scaling. (2) Rapid deployment – deploy code with a single command. (3) Auto-scaling – platform scales resources based on demand. (4) Built-in services – databases, caching, logging, monitoring are available. (5) Cost-effective – pay only for resources used. (6) Focus on code – developers focus on application logic, not infrastructure.

Q3. What is Heroku and how does it work?

Ans: Heroku is a PaaS platform that supports multiple languages (Ruby, Python, Java, Node.js, etc.). Applications are deployed via Git (git push heroku main). Heroku uses Dynos (lightweight Linux containers) to run apps. A Procfile specifies the commands to run the app. Add-ons provide database, caching, and monitoring services.

Q4. What is a Procfile in Heroku?

Ans: A Procfile is a plain text file named 'Procfile' in the root of the application that specifies the commands to run when the app starts. Example: 'web: gunicorn app:app' tells Heroku to start a web process using Gunicorn WSGI server. Different process types (web, worker, clock) can be defined.

Q5. What is the difference between IaaS and PaaS from a developer's perspective?

Ans: With IaaS, the developer manages the operating system, runtime, middleware, application code, and data – only the physical infrastructure is abstracted. With PaaS, the developer only manages application code and data; the platform handles everything else (OS, runtime, middleware, scaling, patching). PaaS is faster for deployment but offers less control than IaaS.

Q6. What is continuous deployment and how does PaaS support it?

Ans: Continuous Deployment (CD) is the practice of automatically deploying every code change that passes testing to production. PaaS platforms support CD through Git integration, webhooks, and CI/CD pipeline tools. For example, connecting a GitHub repository to Heroku enables automatic deployment on every push to the main branch.

Q7. What are environment variables and why are they important in PaaS deployments?

Ans: Environment variables are key-value pairs stored outside the application code to configure runtime behavior. They are used to store sensitive information like database credentials, API keys, and secret keys securely. In PaaS (e.g., Heroku), they are set via the dashboard or CLI (heroku config:set KEY=value) and accessed in code via os.environ in Python or process.env in Node.js.

Q8. What is scalability and how does PaaS handle it?

Ans: Scalability is the ability of an application to handle increased load by adding resources. PaaS handles: (1) Horizontal scaling (scaling out) – adding more instances/dynos/workers automatically or manually. (2) Vertical scaling (scaling up) – increasing the size/resources of each instance. Auto-scaling in PaaS adjusts instance count based on CPU usage, request rate, or custom metrics.

Q9. What is a web framework and which ones are popular for PaaS deployment?

Ans: A web framework provides tools, libraries, and conventions for building web applications. Popular frameworks for PaaS: Python – Flask (micro) and Django (full-featured); Node.js – Express; Java – Spring Boot; Ruby – Ruby on Rails; PHP – Laravel. These frameworks are natively supported by PaaS platforms and have clear deployment guides.

Q10. What steps are involved in deploying a Flask app on Google App Engine?

Ans: Steps: (1) Create Flask app in main.py. (2) Create app.yaml with runtime: python39 and url handlers. (3) Create requirements.txt with Flask and gunicorn. (4) Install Google Cloud SDK. (5) Run 'gcloud auth login' and 'gcloud config set project PROJECT_ID'. (6) Run 'gcloud app create --region=REGION'. (7) Run 'gcloud app deploy'. (8) Access at https://PROJECT_ID.appspot.com.

Assignment 7: Custom Application (Mini Project) Using Salesforce Cloud

Key Oral Questions & Answers

Q1. What is Salesforce and what type of cloud service is it?

Ans: Salesforce is a cloud-based Customer Relationship Management (CRM) platform delivered as Software as a Service (SaaS). It provides tools for sales automation, customer service, marketing, analytics, and application development. The Salesforce platform (Force.com / Salesforce Platform) also acts as a PaaS for building custom business applications.

Q2. What is a Salesforce Object?

Ans: In Salesforce, an Object is equivalent to a database table. It stores data related to a specific entity (e.g., Account, Contact, Opportunity). Standard Objects are pre-built by Salesforce (Account, Lead, Case). Custom Objects are user-defined objects created to store business-specific data. Each object has Fields (columns) and Records (rows).

Q3. What is Apex in Salesforce?

Ans: Apex is Salesforce's proprietary, Java-like, object-oriented programming language used to execute complex business logic on the Salesforce platform. Apex runs on Salesforce servers and can be used to write triggers, classes, batch jobs, web services, and controllers for Visualforce pages. It is tightly integrated with the Salesforce database (SOQL queries).

Q4. What is SOQL and how is it different from SQL?

Ans: SOQL (Salesforce Object Query Language) is a read-only query language used to retrieve data from Salesforce objects. Syntax: `SELECT field1, field2 FROM ObjectName WHERE condition`. Differences from SQL: SOQL can only query one object at a time (no JOINS in the same way), uses relationship queries for related objects, does not support INSERT/UPDATE/DELETE (DML operations use separate statements).

Q5. What is a Salesforce Trigger?

Ans: A Salesforce Trigger is Apex code that automatically executes before or after specific database events (insert, update, delete, undelete) on a Salesforce object. Example: a trigger on Account that sends an email when a new account is created. Best practice is to have one trigger per object and delegate logic to handler classes.

Q6. What are Visualforce Pages in Salesforce?

Ans: Visualforce is a framework for building custom user interfaces (pages) in Salesforce using HTML-like markup tags (e.g., `<apex:page>`, `<apex:form>`, `<apex:inputField>`). Visualforce pages are backed by Apex controller classes that handle business logic and data operations. They are used to create custom UI beyond the standard Salesforce interface.

Q7. What is Salesforce Lightning and how does it differ from Classic?

Ans: Salesforce Lightning is the modern UI framework for Salesforce, introduced to replace Salesforce Classic. It includes Lightning Experience (new UI), Lightning Components (Aura and LWC – reusable UI components built with JavaScript), App Builder, and improved performance. Classic is the older interface, while Lightning provides a component-based, responsive design model.

Q8. What is a Salesforce Sandbox and why is it used?

Ans: A Salesforce Sandbox is a copy of the production environment used for development, testing, and training. Changes made in sandbox do not affect production data. Types: Developer Sandbox (limited storage, full configuration copy), Developer Pro, Partial Copy, and Full Copy. Best practice is to develop in sandbox and deploy to production using Change Sets or CI/CD tools.

Q9. What is the Salesforce AppExchange?

Ans: AppExchange is Salesforce's online marketplace for pre-built applications, components, and consulting services built on the Salesforce platform. It offers free and paid apps that extend Salesforce functionality. Examples: DocuSign for electronic signatures, Conga for document generation. It is similar to an app store for enterprise CRM applications.

Q10. What is a Salesforce Flow?

Ans: Salesforce Flow (formerly Process Builder + Flow Builder) is a point-and-click automation tool that allows users to automate business processes without writing code. Flows can collect data from users, perform record operations (create, update, delete), call Apex code, and send notifications. Screen Flows provide UI for user interaction, while Auto-launched Flows run in the background.

Assignment 8: Firebase Authentication, GAE Standard Environment & Google Cloud Datastore

Key Oral Questions & Answers

Q1. What is Firebase Authentication?

Ans: Firebase Authentication is a backend service provided by Google that simplifies user authentication. It supports multiple sign-in methods: email/password, phone number, Google, Facebook, Twitter, GitHub, and anonymous sign-in. It provides SDKs for web, Android, and iOS. Firebase Auth issues JWT (JSON Web Tokens) upon successful authentication, which can be verified on the server side.

Q2. What is a JSON Web Token (JWT)?

Ans: JWT (JSON Web Token) is an open standard (RFC 7519) for securely transmitting information between parties as a JSON object. A JWT consists of three parts: Header (algorithm type), Payload (claims – user data, expiration time), and Signature (used to verify the token's integrity). Firebase Authentication issues JWTs as ID tokens, which the server verifies using Firebase Admin SDK.

Q3. What is Google Cloud Datastore?

Ans: Google Cloud Datastore (now called Firestore in Datastore mode) is a NoSQL document database service provided by Google Cloud. It is highly scalable, fully managed, and requires no server administration. Data is stored as Entities (similar to rows), organized into Kinds (similar to tables), and identified by Keys. It supports transactions, ACID properties, and powerful query capabilities.

Q4. What is the difference between Firebase Realtime Database and Firestore?

Ans: Firebase Realtime Database stores data as a single large JSON tree, suitable for simple data with frequent small updates (e.g., chat messages). Firestore (Cloud Firestore) stores data as collections of documents with richer data structures, better querying, offline support, and scales better for complex applications. Firestore is the recommended option for new applications.

Q5. How does Firebase Authentication integrate with GAE?

Ans: Integration steps: (1) User signs in via Firebase Auth on the frontend (web SDK). (2) Firebase issues an ID token (JWT). (3) Frontend sends the ID token in the Authorization header of HTTP requests to GAE. (4) GAE backend verifies the token using Firebase Admin SDK (`firebase_admin.auth.verify_id_token()`). (5) If valid, the server processes the request; otherwise returns 401 Unauthorized.

Q6. What is the Firebase Admin SDK?

Ans: Firebase Admin SDK is a server-side library that provides privileged access to Firebase services. It allows backend applications (running on GAE, Cloud Functions, or any server) to: verify Firebase ID tokens, create and manage users, send push notifications via FCM, read and write Firestore/Realtime Database data. Available for Python, Java, Node.js, Go, and C#.

Q7. What are the steps to retrieve, verify, and store user credentials using Firebase and GAE?

Ans: Steps: (1) Set up Firebase project and enable Authentication. (2) Create GAE app. (3) Frontend: integrate Firebase Web SDK, implement sign-in UI (email/password). (4) On successful sign-in, get ID token: `firebase.auth().currentUser.getIdToken()`. (5) Send token to GAE backend in Authorization header. (6) GAE: verify token using Firebase Admin SDK. (7) Extract user UID/email from decoded token. (8) Store/retrieve user data in Cloud Datastore using the UID as the entity key.

Q8. What is the Cloud Datastore API and how do you use it in Python on GAE?

Ans: The Cloud Datastore API allows programmatic access to Datastore. In Python, use the 'google-cloud-datastore' library. To store an entity: create a Client, create a Key with Kind and ID, create an Entity, set properties, and call `client.put(entity)`. To retrieve: use `client.get(key)` or run a Query using `client.query(kind='User')`. The GAE Standard environment provides built-in Datastore access.

Q9. What is the difference between authentication and authorization?

Ans: Authentication is the process of verifying the identity of a user (proving who you are). Methods: password, biometrics, OTP, OAuth. Authorization is the process of determining what an authenticated user is allowed to do (what resources/actions they can access). Example: Firebase Auth handles authentication; checking if a user has admin role to access admin pages is authorization.

Q10. What are environment variables and how are they used in GAE for Firebase integration?

Ans: Environment variables store sensitive configuration like Firebase service account credentials, API keys, and project IDs outside the source code. In GAE, they are defined in app.yaml under the 'env_variables' section or as runtime environment variables. The Firebase Admin SDK is initialized using a service account JSON file or GOOGLE_APPLICATION_CREDENTIALS environment variable pointing to the service account file.

General Cloud Computing – Common Oral Questions

Frequently Asked Theory Questions & Answers

Q1. What are the essential characteristics of cloud computing according to NIST?

Ans: NIST defines 5 essential characteristics: (1) On-demand self-service – users can provision resources without human interaction. (2) Broad network access – accessible over the network from various devices. (3) Resource pooling – provider's resources are pooled to serve multiple consumers (multi-tenancy). (4) Rapid elasticity – resources can scale up/down quickly based on demand. (5) Measured service – resource usage is monitored and reported (pay-per-use).

Q2. What are the cloud deployment models?

Ans: Four deployment models: (1) Public Cloud – services offered over the internet by third-party providers; shared infrastructure; examples: AWS, GCP, Azure. (2) Private Cloud – infrastructure operated exclusively for a single organization; on-premises or hosted; example: OpenStack. (3) Community Cloud – shared by organizations with common concerns. (4) Hybrid Cloud – combination of two or more deployment models, allowing data and applications to move between them.

Q3. What are the three cloud service models?

Ans: Three service models: (1) IaaS (Infrastructure as a Service) – provides virtualized computing infrastructure: VMs, storage, networks. User manages OS and above. Examples: AWS EC2, Google Compute Engine. (2) PaaS (Platform as a Service) – provides a platform for developing and deploying applications. User manages application and data. Examples: GAE, Heroku. (3) SaaS (Software as a Service) – delivers software applications over the internet. Examples: Gmail, Salesforce, Office 365.

Q4. What is virtualization and its role in cloud computing?

Ans: Virtualization is the technology that creates virtual (software-defined) versions of physical hardware resources like servers, storage, and networks. It enables multiple virtual machines to run on a single physical machine using a Hypervisor (e.g., KVM, VMware, Xen). It is the foundational technology of cloud computing, enabling resource pooling, isolation, multi-tenancy, and efficient hardware utilization.

Q5. What is a hypervisor and what are its types?

Ans: A Hypervisor (Virtual Machine Monitor – VMM) is software/firmware that creates and manages virtual machines on a physical host. Type 1 (Bare-metal) hypervisors run directly on hardware without a host OS; more efficient; examples: VMware ESXi, Microsoft Hyper-V, Xen. Type 2 (Hosted) hypervisors run on a host OS; easier to set up; examples: VMware Workstation, VirtualBox, KVM (though KVM is often classified as Type 1).

Q6. What is auto-scaling in cloud computing?

Ans: Auto-scaling is the ability of a cloud system to automatically increase or decrease compute resources based on current demand. It ensures performance during high traffic by scaling out (adding instances) and reduces cost during low traffic by scaling in (removing instances). Auto-scaling policies can be based on CPU usage, memory, request count, custom metrics, or scheduled time windows.

Q7. What is load balancing in cloud computing?

Ans: Load balancing is the process of distributing incoming network traffic across multiple servers/instances to ensure no single server is overwhelmed, improving availability and reliability. Cloud load balancers (e.g., AWS ELB, GCP Load Balancer) operate at Layer 4 (TCP/UDP) or Layer 7 (HTTP/HTTPS). They perform health checks and route traffic only to healthy instances.

Q8. What is SLA (Service Level Agreement) in cloud computing?

Ans: An SLA is a formal contract between a cloud provider and customer that defines the expected level of service. It specifies: availability/uptime guarantee (e.g., 99.9% SLA = ~8.76 hrs downtime/year),

performance metrics (response time, throughput), support response times, penalties for SLA violations, data security and privacy commitments, and disaster recovery objectives (RTO, RPO).

Q9. What is data security in cloud computing?

Ans: Data security in cloud involves: (1) Encryption – data encrypted at rest (AES-256) and in transit (TLS/SSL). (2) Identity and Access Management (IAM) – controlling who can access what resources. (3) Multi-factor authentication. (4) Network security – firewalls, VPC, VPN. (5) Compliance – meeting regulatory standards (GDPR, HIPAA, ISO 27001). (6) Backup and disaster recovery. (7) Audit logging and monitoring.

Q10. What is the CAP theorem in distributed systems/cloud?

Ans: CAP Theorem (Brewer's Theorem) states that a distributed data store can only guarantee at most TWO of the following three properties simultaneously: (1) Consistency – all nodes see the same data at the same time. (2) Availability – every request receives a response. (3) Partition Tolerance – system continues to operate despite network partitions. Cloud databases make trade-offs: e.g., Cassandra (AP), HBase (CP), traditional RDBMS (CA but not partition-tolerant).

Best of Luck for Your External Oral Exam!

Savitribai Phule Pune University | TE IT (2019 Course) | 314458 Cloud Computing Lab